

Prioritized Progress Report and Status

1 Completed or Covered at Least Once

You have made significant progress on key areas of data structures and algorithms. Here's a summary of what you've covered:

- **Linked Lists:** Covered single, double, and circular linked lists.
- **Stacks and Queues:** Implemented and studied fundamental operations.
- **Binary Trees:** Covered binary tree structures and their traversals.
- **Binary Search Trees:** Studied and implemented insertions, deletions, and traversals.
- **Matrix Multiplication:** Covered Strassen's Algorithm.
- **Sorting Algorithms:** Implemented Quick Sort, Merge Sort, and Heap Sort.
- **Binary Search:** Studied and implemented binary search on sorted arrays.
- **Red-Black Trees:** Covered insertion, deletion, and all balancing operations, with full implementations in C++ (not Rust for some fixups).
- **Graphs:** Implemented adjacency lists and adjacency matrices, with some additional graph work.
- **HashMaps:** Implemented open-addressing hash tables in C and Rust, using double hashing and quadratic probing, with performance analytics.

2 Foundational Work Covered

- **Automata and Regular Expressions:** Explored foundational concepts, including finite automata and regular expressions.

3 Skimmed but Not Fully Explored (May Re-visit Later)

- **Chapter 5: Probabilities:** Skimmed; probability theory is useful but not immediately essential for your current focus.
- **Chapter 8: Sorting in Linear Time:** Covered some basic ideas of Counting Sort, Radix Sort, and Bucket Sort.
- **Chapter 9: Medians and Order Statistics:** Skimmed; revisiting may be useful for deeper optimization techniques.

4 To-Do List (Prioritized for Progress)

4.1 Core Data Structures and Algorithms (Highest Priority)

- **Breadth-First Search (BFS):** Next logical step based on your graph work. BFS is crucial for graph traversal and will lead into other algorithms like Dijkstra's.
- **Depth-First Search (DFS):** Follow BFS with DFS to complete core graph traversal techniques.
- **Dijkstra's Algorithm:** Build on BFS and DFS to implement this shortest-path algorithm.
- **Kruskal's and Prim's Algorithms:** After DFS, tackle these minimum spanning tree algorithms. They pair well with your understanding of graphs and Union-Find.
- **AVL Trees:** You have done red-black trees; AVL trees are next in the balancing tree family. A natural progression for tree structures.
- **Union-Find (Disjoint Sets):** Essential for graph algorithms like Kruskal's. Manageable with your current skills.

4.2 Dynamic Programming (Medium Priority)

- **Dynamic Programming (Knapsack, Fibonacci):** After graph algorithms, shift focus to dynamic programming, which is key for solving optimization problems. Start with basic problems like the knapsack problem and Fibonacci sequence.

4.3 Additional Data Structures (Medium Priority)

- **B-Trees**: Important for databases and file systems. A good addition after AVL trees.
- **Tries**: Useful for solving string problems efficiently.

4.4 Advanced Data Structures (Optional but Good to Know)

- **Fibonacci Heaps**: Useful for optimizing graph algorithms, especially Dijkstra's.
- **van Emde Boas Trees**: A niche structure; useful in specific cases, but can be deferred for now.

4.5 Advanced Algorithms (Medium Priority)

- **A* Search Algorithm**: Builds on Dijkstra's algorithm, but with heuristics for better performance in pathfinding.
- **String Matching Algorithms (KMP, Rabin-Karp)**: Important for text search problems, but can be tackled after core graph algorithms and dynamic programming.

5 Overall Recommendation

1. Continue with **BFS** and **DFS**, as they are foundational for graph traversal.
2. Move to **Dijkstra's Algorithm** and then tackle **Kruskal's** and **Prim's**.
3. Implement **AVL Trees** and **Union-Find** (Disjoint Sets) to strengthen your grasp of balancing and graph-based data structures.
4. Once those are completed, shift focus to **Dynamic Programming** (Knapsack, Fibonacci) to deepen your problem-solving skills.
5. As a lower priority, explore **B-Trees** and **Tries** to broaden your data structures toolkit.
6. Finally, explore advanced structures like **Fibonacci Heaps** and **van Emde Boas Trees**, but only if needed for specific applications.